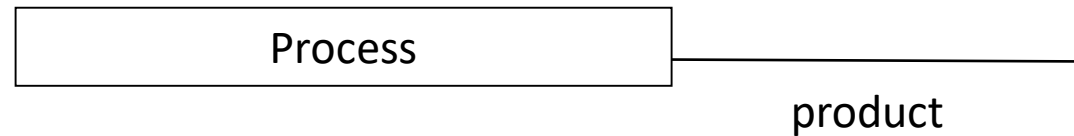


Requirement Engineering

Requirement Engineering

- Requirement engineering is about defining the product properties before starting development





Requirements



Properties

Without RE

- Product properties are unclear
- Testing is not feasible

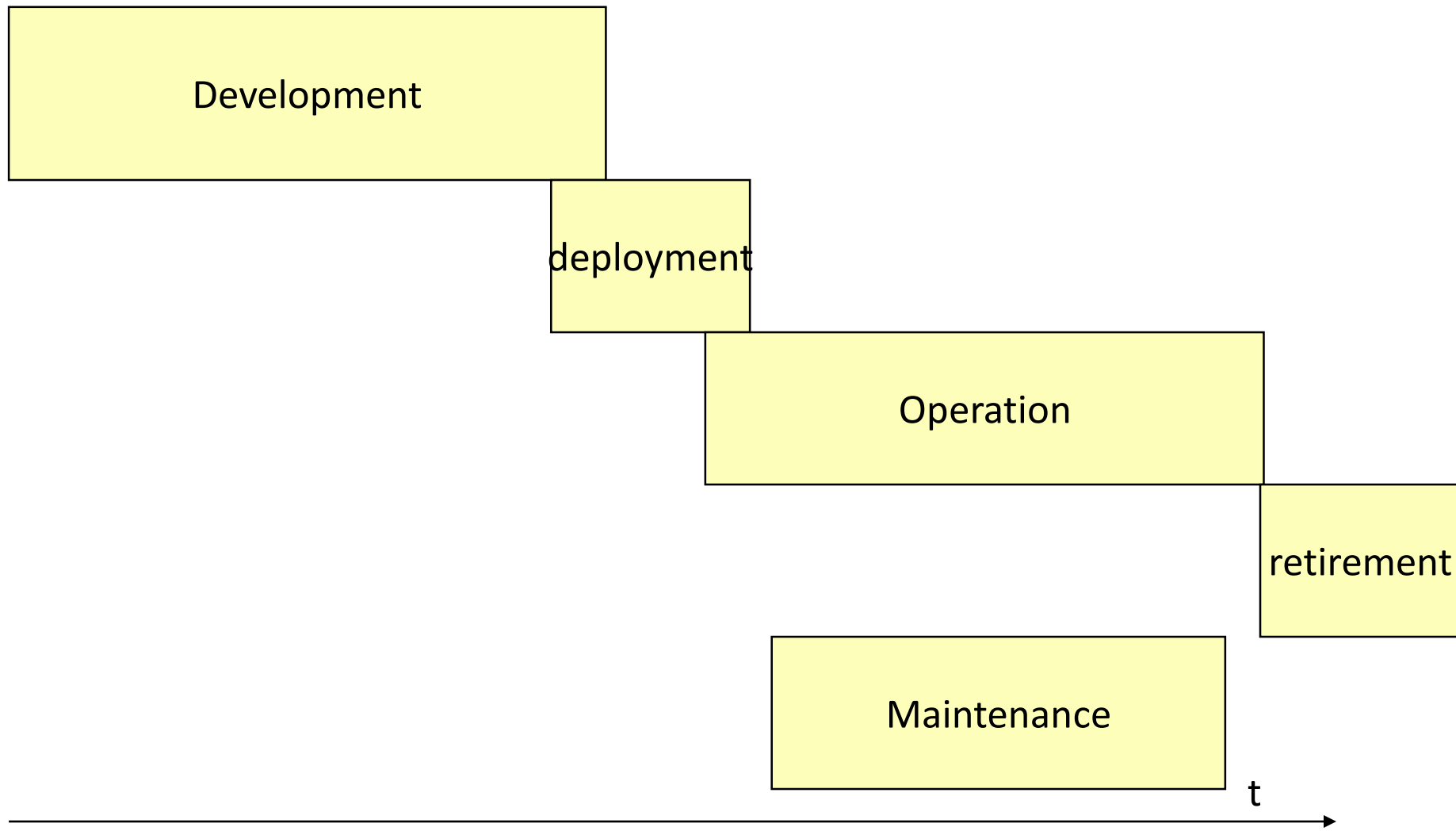
Product properties

- Functional
 - what should the program do
- Non-functional
 - Modalities applied to functions offered
 - Reliability
 - Usability
 - Performance
 - Maintainability
 - Security
 - ...

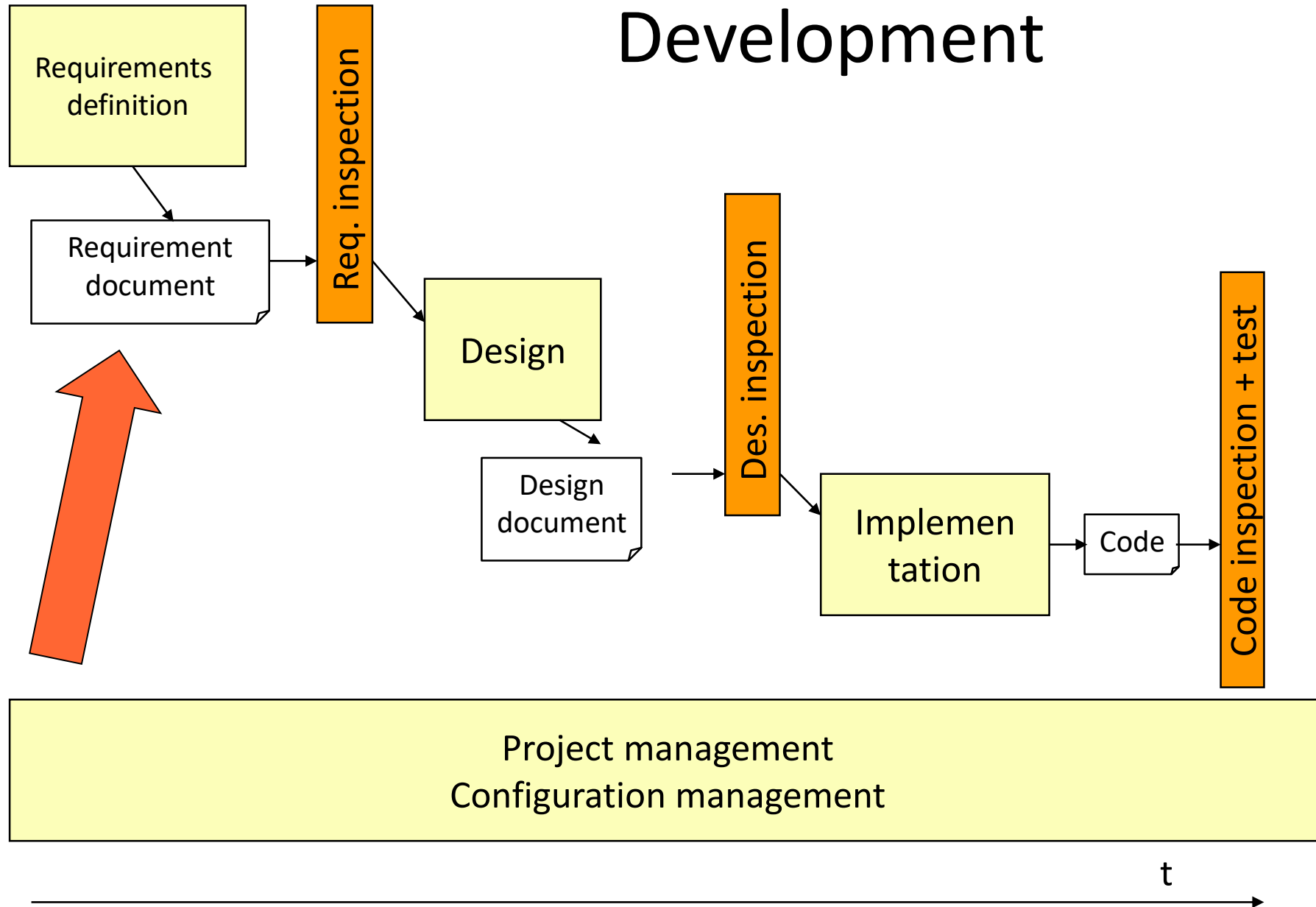
Outline

- Concepts and definitions
- Techniques for formalization
- Techniques for elicitation
- Techniques for V and V

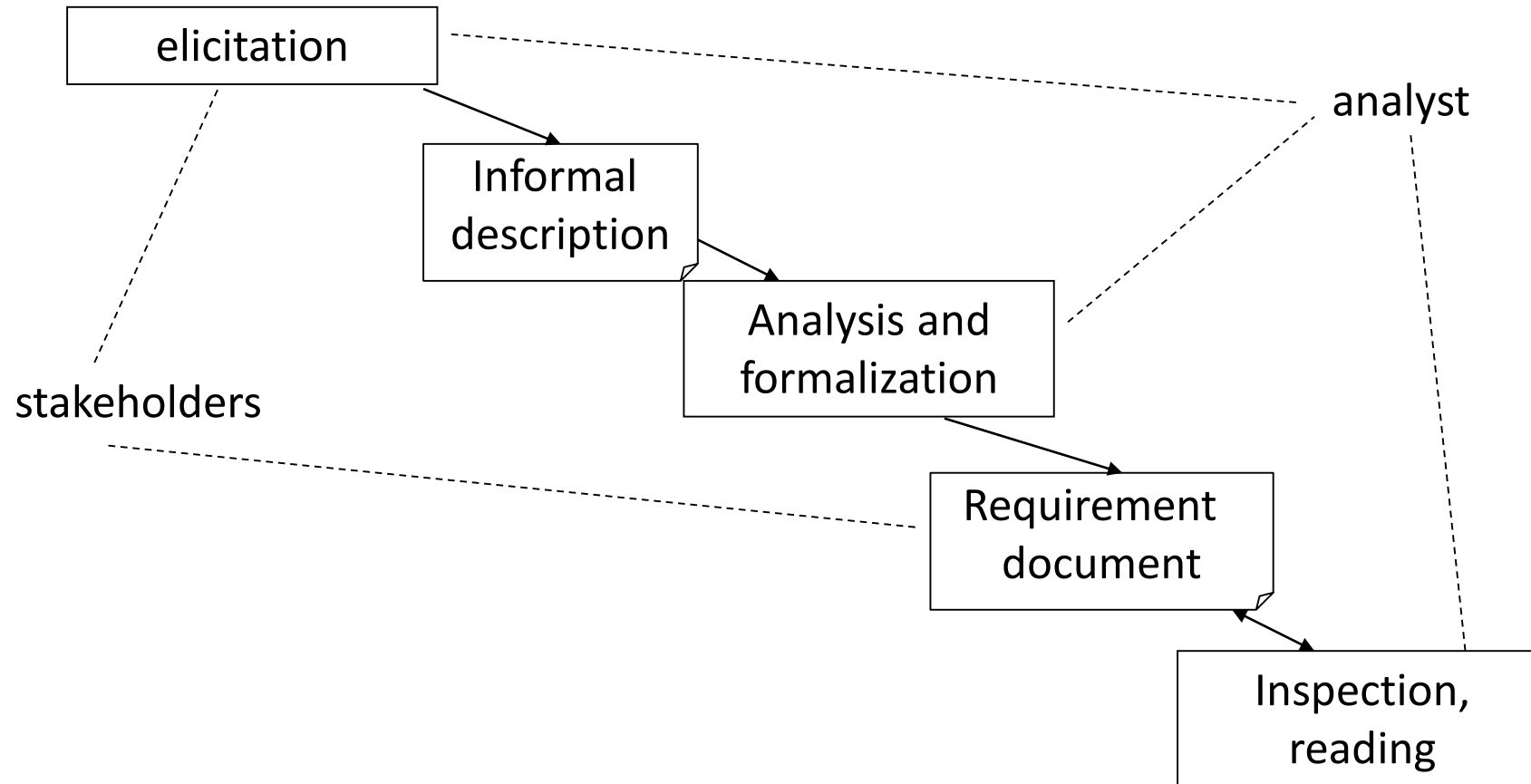
Main Phases



Development



Requirement engineering



Requirement

- Description of product property
 - Functional – not functional
- The final product may, or may not, have properties that match the requirements



Requirements



Properties

Requirements vs. properties

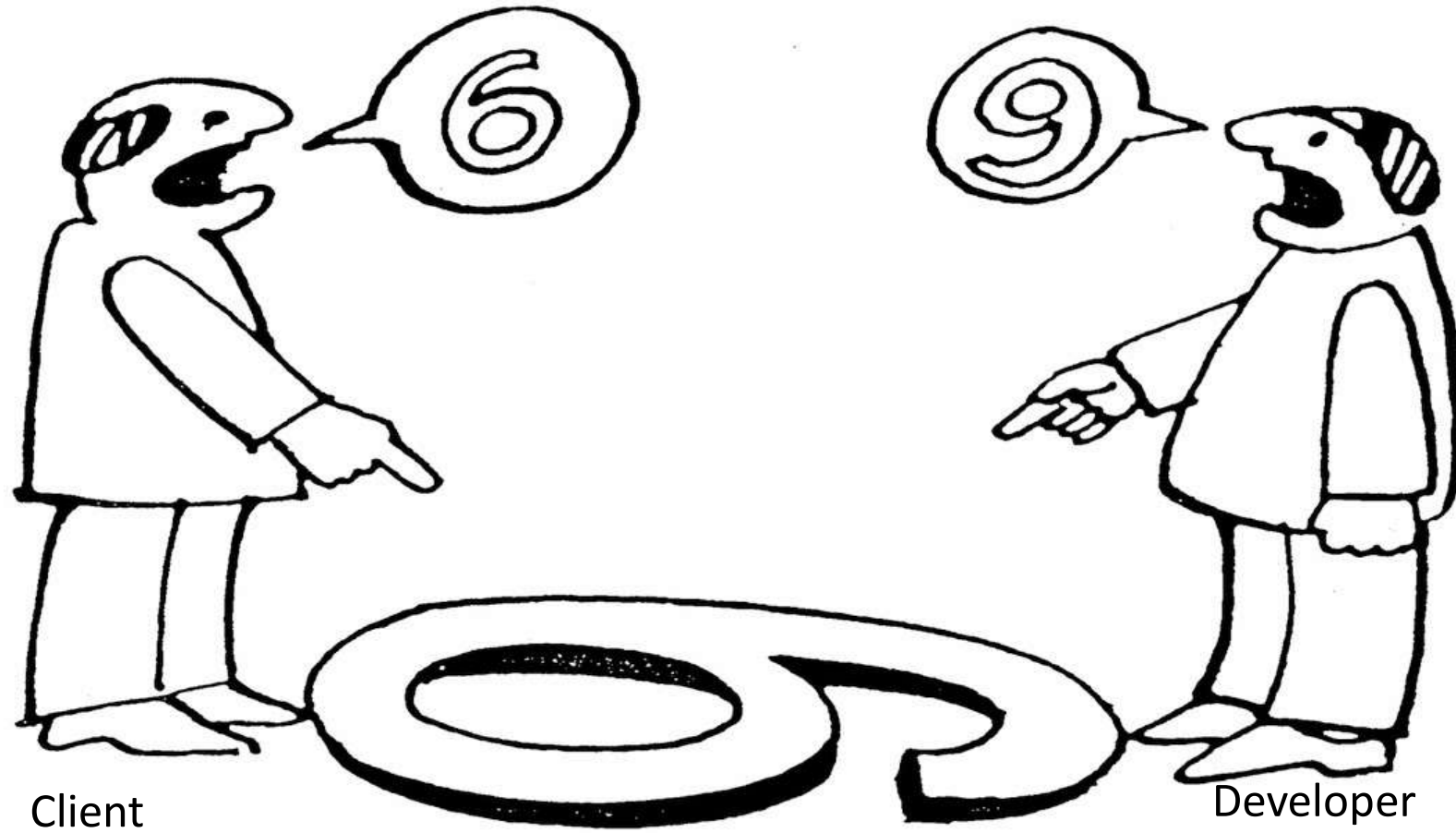


Requirements are requests, that may or may not become properties of the product

- Ex.
- Requirement: response time of all functions $< 0,5$ sec
- Property: response time of functions between 2secs and 10 secs

Starting point

- Usually an informal description by a client or a potential user to the developer



Requirements - informal

- A POS (Point-Of-Sale) system is a computer system typically used to manage the sales in retail stores. It includes hardware components such as a computer, a bar code scanner, a printer and also software to manage the operation of the store.
- The most basic function of a POS system is to handle sales.

Requirements - informal

- Problems arise when requirements are not precisely stated.
- Ambiguous requirements may be interpreted in different ways by developers and users.
 - 'manage sales' and 'handle sales' do mean the same thing?
 - What is a sale? Exchange of items? Payment?

Completeness and consistency

- In principle, requirements should be both complete and consistent.
 - Complete
 - They should include descriptions of all features required.
 - Consistent
 - There should be no conflicts or contradictions in the descriptions of the system features.
- In practice, it is quite utopistic to produce a complete and consistent requirements document.

Requirements - defects

- Omission/ incompleteness
- Incorrect Fact
- Inconsistency/contradiction
- Ambiguity
- Extraneous Information
 - Overspecification (design)
- Redundancy

Requirements - defects

- Omission/ incompleteness
 - Log of daily sales should be made and kept for 5 years (omitted)
- Incorrect Fact
 - Sales are subject to 22% VAT (page 3 of document)
 - No, VAT depends on item, 10% for some, 22% for others
- Inconsistency/contradiction
 - Sales are subject to 21% VAT (page 10 of document)
- Ambiguity
 - 'handle sales' and 'manage sales' are same thing or not?
- Extraneous Information
 - The sales should be stored using a hash table
- Redundancy
 - Sales are subject to 22% VAT (page 3 of document)
 - Sales are subject to 21% VAT (page 10 of document)

Techniques - formalization

- Stakeholders
- Business model
- Personas, stories
- Context diagram and interfaces
- Requirements, F, NF
- Table of rights
- Scenarios, sequence diagrams
- Use cases
- Glossary
- System design

Stakeholder

- Basic idea: ask the client an (informal) description of the product to build
- Who exactly is the 'client'?

Stakeholder

- Role person company or system that is directly or indirectly involved in the project and who may affect or get affected by the outcome of the project
- User
 - Uses the application
 - Can include different user profiles
- Buyer/commissioner
 - Pays for the application
- Administrator
- Analyst
 - Expert in requirement engineering and or in the domain
- Developer

Stakeholders - example

- Point Of Sale (POS) in a supermarket
- User
 - Cashier at POS (profile 1)
 - Supervisor, inspector (profile 2)
 - Customer at POS (indirectly through cashier)
- Administrator
 - POS application administrator (profile 3)
 - IT administrator (profile 4)
 - Manages all applications in the supermarket
 - Security manager (profile 5)
 - Responsible for security issues
 - DB administrator (profile 6)
 - Manages DBMSs on which applications are based
- Buyer
 - CEO and/or CTO or CIO of supermarket

Stakeholders

- Listing the relevant stakeholders is essential to consider relevant points of view (and therefore relevant requirements) for an application

Business model

- How the application is funded?
- Options
 - An organization pays for development and operation and maintenance, users use it for free
 - Ex: University applications. University pays for all, users (students, employees) use them for free
 - An organization pays for development and operation, users pay to use the application
 - Ex: MS windows, Netflix, ..
 - Many payment options exist: pay per use, pay per period, pay per volume (ex cloud services)
 - Users don't pay but must watch ads, organization is paid for each ad shown
 - Ex: Instagram, Facebook, meteo web sites, ..

Business model

- The business model involves some set of requirements
 - Ex: if users need to pay, payment functions are needed
 - Ex: if payment model is per use/per period/per volume , functions to monitor usage rights are needed
 - Ex: if users must see ads, functions to show ads are needed

Stories and personas

- A technique to informally define what the application should do
- Best suited for mass market applications where 'end user' has little meaning
- AKA profiling / segmentation in marketing, adapted to software products

Profiling / segmentation

- Characterize people under a number of dimensions, to better define and sell targeted products / services

Demographic profile

- Age
- Gender
- Ethnicity
- Income
- Level of education
- Religion
- Profession/role in a company

Psychographic profile

- Personality traits
- Hobbies
- Life goals
- Values
- Beliefs
- Lifestyles

Geographic profile

- Country
- Region
- City
- Postal code

Ex market segments

In garments business, segments could be:



Profiling in e-commerce

Web site behaviour

- Number of sessions website
- Number of pages visited
- Time spent on site
- URLs visited
- Page types visited
- Shopping cart value
- Referral source
- Inactivity

Web site behaviour - 2

- First time visitor
- Returning visitor
- Returning customer

Use of profiles

- First time visitor
 - Offer x% discount on first order
- Returning visitor
 - Remind of x% discount
- Returning customer
 - Propose products similar to ones previously bought

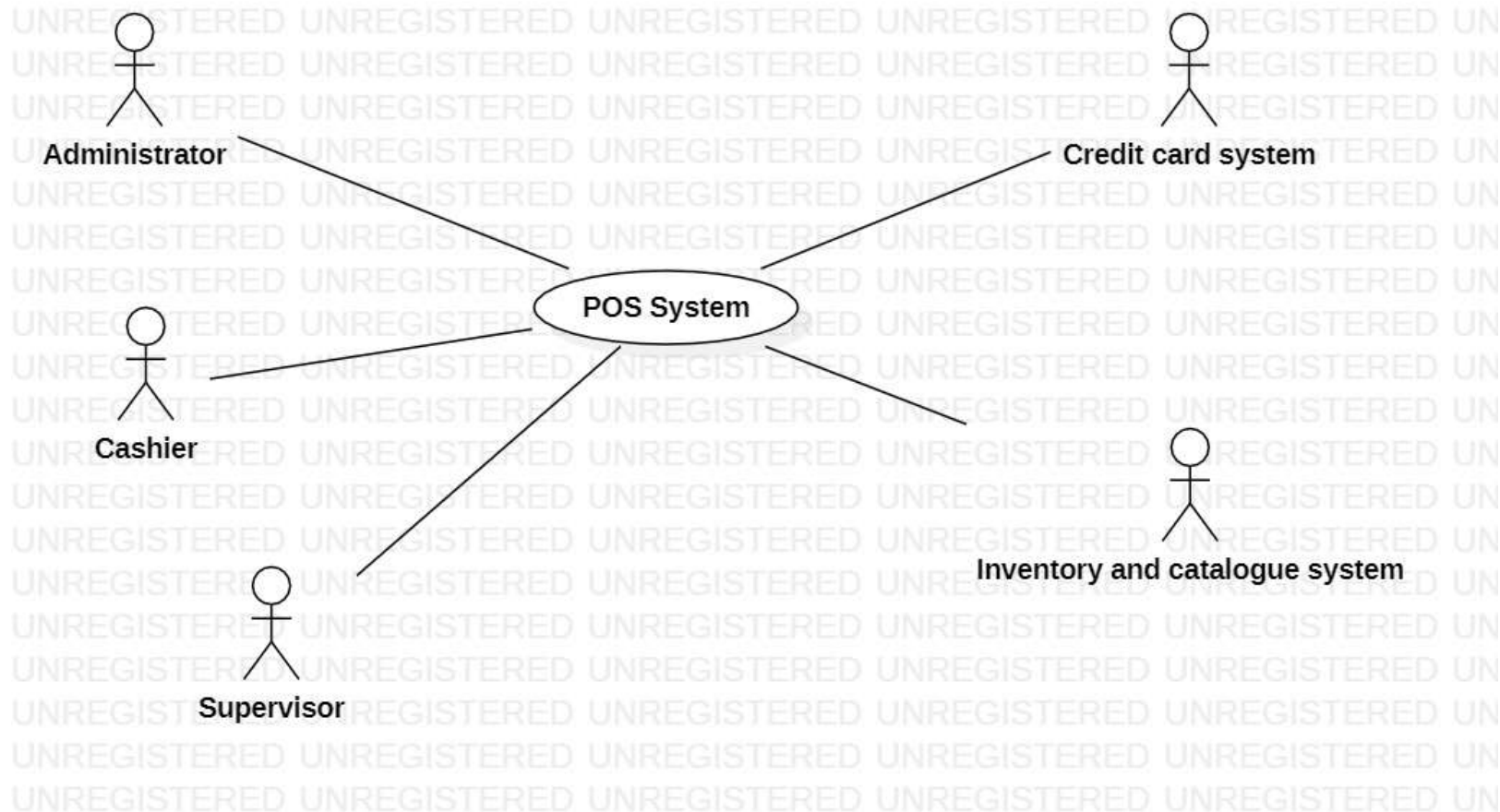
Context diagram

- To understand 'in' and 'out'

Context diagram

- Defines what is **inside** the application to be developed, what is **outside**
 - Entity outside = actor
 - $\{\text{actor}\} \subset \{\text{stakeholder}\}$
 - Other systems/subsystems/applications
 - Human users (can specialize in one or more personas)
- Defines **interfaces** between inside and outside

Context diagram



Interfaces

- With cashier / administrator
 - Physical level: Screen, keyboard
 - Logical: GUI (to be described)
- With product
 - Physical level: laser beam (bar code reader)
 - Logical level: bar code
- With credit card system
 - Physical level: internet connection
 - Logical: web services (functions to be described, data exchanged, ex http/REST + json)

Interfaces – tabular form

Actor	Physical interface	Logical interface
Cashier	Screen, keyboard	Graphical User Interface (to be described, ex slide 39)
Product	Laser beam	ReadBarCode (to be specified, ex slide 37,38)
Credit card system	Internet connection	API description (ex https://developer.visa.com/docs for ViSA APIs)
Administrator	Screen, keyboard	Graphical User Interface + command line interface

Interface specification

- Three types of interface may have to be defined
 - User interfaces, GUIs
 - Procedural interfaces;
 - Data exchanged;
- Formal notations are an effective technique for interface specification.

Procedural interface description

```
interface PrintServer {  
  
    // defines an abstract printer server  
    // requires:      interface Printer, interface PrintDoc  
    // provides: initialize, print, displayPrintQueue, cancelPrintJob, switchPrinter  
  
    void initialize ( Printer p ) ;  
    void print ( Printer p, PrintDoc d ) ;  
    void displayPrintQueue ( Printer p ) ;  
    void cancelPrintJob (Printer p, PrintDoc d) ;  
    void switchPrinter (Printer p1, Printer p2, PrintDoc d) ;  
} //PrintServer
```

Data interface (XML)

```
<food>  
<name>Belgian Waffles</name>  
<price>$5.95</price>  
<description>  
two of our famous Belgian Waffles with plenty of real maple syrup  
</description>  
<calories>650</calories>  
</food>
```

```
<food>  
<name>Strawberry Belgian Waffles</name>  
<price>$7.95</price>  
<description>  
light Belgian waffles covered with strawberries and whipped cream  
</description>  
<calories>900</calories>  
</food>
```

GUI interface

- Sketch of interface, typically built with GUI builder

